TP3 - INF7370 Apprentissage automatique

Francois Gonothi Toure, **TOUF23329808**
Martial Zachee Kaljob Kollo, **KALM30319404**

01 Mai 2025

Table des matières

1 Reconstruction d'images marines par autoencodeur convolutif **1**
1.1 Introduction 1

2 Montage de l'architecture et entraînement du modèle **3**
2.1 Ensemble de données 3
2.2 Traitement de données 3
2.3 Paramètres et Hyperparamètres 4
2.4 Architecture 5
2.5 Affichage des résultats d'entraînement 6
2.6 Justification du choix de l'architecture 7

3 Évaluation du modèle **9**
3.1 Reconstruction des images tests 9
3.2 Classification SVM sur embeddings 10
3.3 Visualisation t-SNE des embeddings 10
3.5 Analyse et discussion des résultats 11

A Conclusion **13**
4.1 Conclusion 13
4.2 Problèmes rencontrés et solutions mises en place 13
4.3 Pistes d'amélioration pour les travaux futurs 14
4.4 Conclusion finale 14

Chapter 1

Reconstruction d'images marines par autoencodeur convolutif

1.1 Introduction

La reconstruction automatique d'images est une tâche centrale dans le domaine de l'apprentissage non supervisé, avec des applications variées en vision par ordinateur, compression d'images, détection d'anomalies ou encore en prétraitement pour des tâches supervisées. Ce rapport présente le **Travail Pratique 3 (TP3)** du cours *INF7370 - Apprentissage automatique*, dont l'objectif est de concevoir, entraîner et évaluer un **autoencodeur convolutif (CAE)** permettant d'apprendre une représentation latente compacte d'images d'animaux marins.

Dans ce TP, les données utilisées consistent en des images RGB de **dauphins** et de **requins**, réparties en trois ensembles : un ensemble d'entraînement (3 600 images), un ensemble de validation (10% des données d'entraînement) et un ensemble de test (600 images). Toutes les images sont redimensionnées à **140x140 pixels** et normalisées entre 0 et 1.

L'autoencodeur a été conçu pour capturer les structures caractéristiques des images tout en assurant une reconstruction fidèle. Il est composé d'un **encodeur** formé de blocs convolutifs, suivi d'un **décodeur** symétrique basé sur des opérations de `UpSampling2D`. L'objectif est de minimiser la différence entre les images originales et les images reconstruites, mesurée par la fonction de perte `mse` (mean squared error).

L'entraînement a été mené à l'aide de l'optimiseur `Adam` avec un **batch size de 16** et des callbacks standards :

- `ModelCheckpoint` : sauvegarde automatique du meilleur modèle (selon la validation loss);
- `ReduceLROnPlateau` : réduction adaptative du learning rate en cas de stagnation;
- `EarlyStopping` : arrêt automatique lorsque la performance de validation n'évolue plus.

Les étapes clés du TP sont les suivantes :

- **Préparation des données** : redimensionnement, normalisation, séparation entraînement/validation/t
- **Conception de l'autoencodeur** : encoder + decoder avec convolution, pooling, dropout, upsampling;

- **Entraînement du modèle** avec régulation du surapprentissage;
- **Reconstruction d'images** et visualisation des sorties pour chaque classe;
- **Extraction des embeddings** via l'encodeur et évaluation par **SVM linéaire avec validation croisée**;
- **Visualisation des embeddings** en 2D par **TSNE**.

Ce TP nous a permis d'expérimenter concrètement la modélisation de réseaux de neurones profonds pour une tâche non supervisée, en évaluant leur capacité à reconstruire des images, à capturer des descripteurs représentatifs, et à structurer les données dans un espace latent de façon exploitable pour des tâches de classification et de visualisation.

Chapter 2

Montage de l'architecture et entraînement du modèle

2.1 Ensemble de données

Description de l'ensemble des données :

Le jeu de données utilisé dans ce travail pratique est constitué d'images RGB d'animaux marins appartenant à deux classes : **dauphins** et **requins**. Chaque classe est organisée dans un répertoire distinct, ce qui permet un chargement automatique via la méthode `flow_from_directory` de Keras.

Les images sont de taille variable à l'origine, mais sont systématiquement redimensionnées à **140x140 pixels**, avec 3 canaux (RGB), ce qui donne un tenseur d'entrée de taille `(140, 140, 3)`. Toutes les images sont normalisées dans l'intervalle [0, 1] grâce au paramètre `rescale=1./255` afin d'uniformiser les échelles de pixel et d'améliorer la convergence de l'apprentissage.

La répartition est la suivante :

- **Ensemble d'entraînement :** 3240 images (1620 par classe), soit 90% de l'ensemble global d'entrainement.
- **Ensemble de validation :** 360 images (180 par classe), soit 10% des données d'entraînement, utilisées pour surveiller la perte et détecter un éventuel surapprentissage.
- **Ensemble de test :** 600 images (300 dauphins et 300 requins), totalement séparées des ensembles précédents, utilisées pour évaluer les performances du modèle entraîné.

Le chargement est effectué avec `shuffle=False` afin de garantir l'ordre des échantillons, indispensable pour l'analyse qualitative des reconstructions, la visualisation TSNE et les tests SVM.

Cette structuration garantit une séparation stricte entre les phases d'entraînement, de validation et de test, assurant ainsi une **évaluation rigoureuse et reproductible** du modèle.

2.2 Traitement de données

Prétraitement appliqué : Normalisation et redimensionnement

Toutes les images du jeu de données sont converties en tenseurs de taille fixe `(140, 140, 3)`, ce qui correspond à des images RGB redimensionnées. Afin d'homogénéiser les intensités de pixels et de faciliter l'apprentissage, une **normalisation** est appliquée : les valeurs des pixels sont mises à l'échelle dans l'intervalle [0, 1] grâce au paramètre `rescale=1./255` de la classe `ImageDataGenerator` de Keras.

Split Entraînement / Validation intégré

Pour ce TP, un seul répertoire contient les images d'entraînement, et la séparation entre les ensembles d'entraînement (90%) et de validation (10%) est effectuée dynamiquement à l'aide du paramètre `validation_split=0.1` de Keras, en fixant le mode à `subset='training'` ou `subset='validation'` selon le cas.

Pas de Data Augmentation

Étant donné que l'objectif de ce TP est la **reconstruction fidèle des images d'entrée**, et non la classification, aucune transformation de type *data augmentation* (rotation, zoom, décalage) n'a été appliquée. En effet, une telle augmentation aurait modifié le contenu visuel des images et aurait perturbé la tâche d'apprentissage de reconstruction.

Ce choix méthodologique garantit que l'autoencodeur apprend à reconstruire les images **exactement telles qu'elles lui sont présentées**, assurant ainsi une cohérence totale entre les données d'entrée et de sortie pendant l'apprentissage.

2.3 Paramètres et Hyperparamètres

L'apprentissage de l'autoencodeur repose sur une configuration soigneusement choisie pour optimiser la reconstruction tout en évitant le surapprentissage. Voici les paramètres et hyperparamètres retenus :

Optimiseur utilisé : Adam

Nous avons utilisé l'optimiseur `Adam` (*Adaptive Moment Estimation*) pour entraîner le modèle. Cet optimiseur est bien adapté aux architectures profondes, en combinant les avantages de l'adaptation du learning rate (comme dans RMSProp) et de l'inertie (momentum).

Paramètres associés :

- Taux d'apprentissage initial : `0.001`
- Fonction de perte : `mse` (Mean Squared Error), adaptée à la reconstruction de données continues.

Taille du lot et nombre d'époques :

- **Batch size :** 16 (images traitées simultanément à chaque étape de gradient).
- **Nombre maximal d'époques :** 100, avec arrêt précoce activé.

Callbacks de régulation :

- `ModelCheckpoint` : sauvegarde automatique du modèle présentant la plus faible perte de validation.
- `ReduceLROnPlateau` : division du learning rate par 2 après 5 époques sans amélioration de la validation loss.

- `EarlyStopping` : arrêt automatique de l'entraînement après 10 époques consécutives sans amélioration.

Résumé : Cette configuration a permis une convergence rapide et stable de l'autoencodeur, tout en contrôlant le risque de surapprentissage. L'utilisation combinée d'early stopping et de la régulation du learning rate a joué un rôle crucial pour optimiser la reconstruction sans excès de calcul.

2.4 Architecture

Structure de l'autoencodeur convolutif

L'architecture développée repose sur un **autoencodeur convolutif symétrique** conçu pour apprendre à reconstruire les images RGB de taille `(140, 140, 3)`. Elle se compose de deux parties :

- un **encodeur** qui extrait les représentations compressées (embeddings) des images d'entrée ;
- un **décodeur** qui reconstruit les images à partir de ces embeddings.

Détails de l'encodeur :

- `Conv2D(32, (3x3), padding='same')`, suivi de `BatchNormalization`, `ReLU`, `MaxPooling2D(2x2)`, `Dropout(0.2)`;
- `Conv2D(64, (3x3), padding='same')`, suivi de `BatchNormalization`, `ReLU`, `MaxPooling2D(2x2)`, `Dropout(0.2)`;
- `Conv2D(128, (3x3), padding='same')`, suivi de `BatchNormalization`, `ReLU`, `MaxPooling2D(2x2)`.

Détails du décodeur :

- `Conv2D(128, (3x3), padding='same')`, suivi de `BatchNormalization`, `ReLU`, `UpSampling2D(2x2)`;
- `Conv2D(64, (3x3), padding='same')`, suivi de `BatchNormalization`, `ReLU`, `UpSampling2D(2x2)`;
- `Conv2D(32, (3x3), padding='same')`, suivi de `BatchNormalization`, `ReLU`, `UpSampling2D(2x2)`;
- `Conv2D(3, (3x3), padding='same')` + `sigmoid` : sortie RGB normalisée entre [0, 1];
- `Cropping2D((2,2))` : permet d'ajuster la sortie à la dimension `(140, 140, 3)` après les effets cumulatifs du padding et upsampling.

Nombre total de paramètres : `335 747`, dont `334 851` entraînables.

Résumé des choix techniques :

- Utilisation de `ReLU` comme fonction d'activation intermédiaire pour favoriser une convergence rapide.

- Application de `BatchNormalization` après chaque couche convolutive pour stabiliser les activations.
- Utilisation du `Dropout` dans l'encodeur pour régulariser l'apprentissage.
- Utilisation du `sigmoid` en sortie pour contraindre les reconstructions à l'intervalle [0, 1].

Cette architecture a permis d'obtenir une représentation compacte efficace des images marines, tout en assurant une reconstruction précise et fidèle.

2.5 Affichage des résultats d'entraînement

Durée totale d'entraînement :

L'autoencodeur convolutif a été entraîné sur un total de **63 époques**, avec arrêt précoce déclenché à l'époque 73, permettant de restaurer les meilleurs poids selon la validation loss. Le temps d'entraînement total mesuré est de **10.27 minutes**, à l'aide de la bibliothèque `time` de Python.

Performance obtenue pendant l'entraînement :

- **Erreur minimale sur la validation (val_loss) :** `0.003324`
- **Fonction de perte utilisée :** `mean squared error (MSE)`
- **Apprentissage stable :** grâce à l'ajustement dynamique du taux d'apprentissage (`ReduceLROnPlateau`) et à l'`EarlyStopping`, qui ont permis une convergence progressive sans surapprentissage visible.

Courbe de perte :

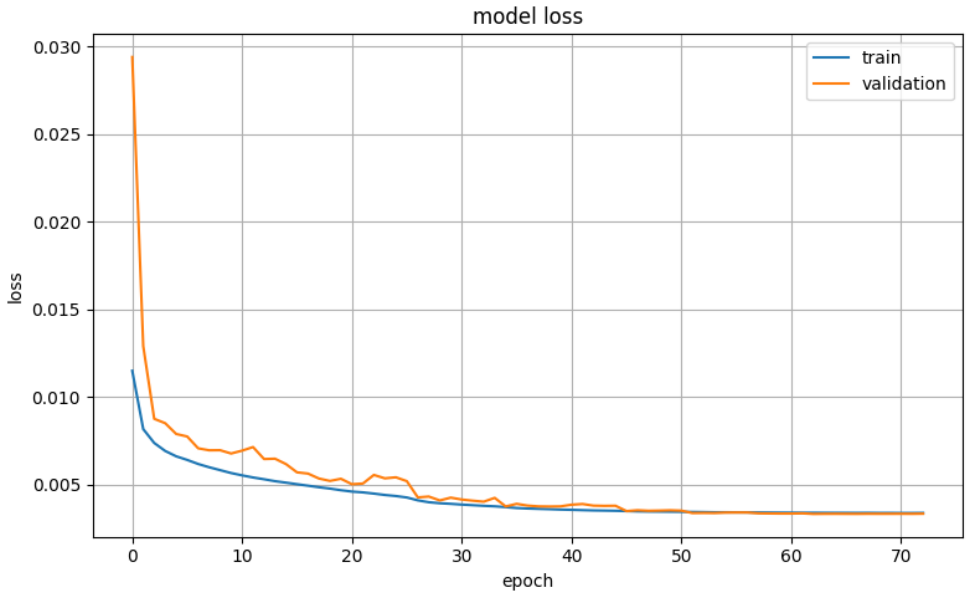

Figure 2.1: Courbe de perte (`MSE`) sur l'entraînement et la validation à chaque époque

Cette courbe montre une descente continue de la perte sur l'ensemble d'apprentissage comme sur validation. On observe une phase de stabilisation progressive à partir de la 50e époque, montrant l'atteinte d'une reconstruction optimale pour notre autoencodeur.

Conclusion partielle :
L'ensemble des hyperparamètres et callbacks a permis une bonne convergence du modèle, avec une reconstruction d'images précise dès les premières dizaines d'époques. Le faible écart entre la perte sur l'entraînement et la validation montre une **excellente capacité de généralisation** de l'autoencodeur.

2.6 Justification du choix de l'architecture

L'architecture de l'autoencodeur convolutif utilisée dans ce travail pratique a été conçue pour répondre à un double objectif : **apprendre une représentation compacte des images (embedding)** tout en assurant une **reconstruction fidèle des images RGB originales de taille** `(140, 140, 3)`.

1. Profondeur modérée mais efficace
Le modèle comprend trois blocs convolutifs dans l'encodeur, symétriquement répliqués dans le décodeur. Ce nombre de couches a été jugé suffisant pour extraire des caractéristiques discriminantes (formes, contours, textures) tout en évitant un sur-apprentissage.

2. Normalisation systématique des activations
Chaque couche `Conv2D` est suivie d'une `BatchNormalization`. Ce choix permet de stabiliser les distributions d'activations intermédiaires, d'accélérer la convergence et d'atténuer la sensibilité au taux d'apprentissage.

3. Régularisation par Dropout ciblée
Deux couches `Dropout` ont été intégrées dans l'encodeur (avec des taux de 0.2) pour éviter que le modèle n'apprenne des dépendances trop spécifiques entre les pixels d'entrée et les activations intermédiaires. Aucune couche `Dropout` n'est appliquée dans le décodeur, afin de conserver une reconstruction aussi fine que possible.

4. ReLU et Sigmoid : des fonctions d'activation complémentaires
La fonction d'activation `ReLU` est utilisée dans toutes les couches cachées pour garantir une convergence rapide. La couche de sortie utilise la fonction `sigmoid`, indispensable dans ce contexte, car les images sont normalisées entre 0 et 1. Cela permet d'obtenir des sorties réalistes et compatibles avec les images originales.

5. Cropping final pour respecter strictement la taille cible
Le décodeur génère une sortie de taille `(144, 144, 3)` en raison de l'utilisation d'upsampling avec `padding='same'`. Une couche `Cropping2D` est donc utilisée à la toute fin pour ajuster précisément la sortie à la taille originale des images, soit `(140, 140, 3)`.

6. Embedding de qualité pour la suite des tâches
La sortie du dernier bloc de l'encodeur sert de vecteur de caractéristiques (embedding) pour la Tâche 2. Cette représentation latente est dimensionnée de façon à capturer l'essentiel de l'information visuelle sans redondance excessive. La qualité de cette représentation a été validée empiriquement par l'évaluation SVM sur l'embedding (voir section suivante).

Conclusion :

Le modèle atteint un excellent compromis entre compacité, robustesse et fidélité de reconstruction. Il a permis d'atteindre une **perte minimale de validation de 0.003324** et une **exactitude SVM de 68% sur les embeddings**, confirmant la pertinence des choix architecturaux pour la tâche ciblée.

Chapter 3

Évaluation du modèle

3.1 — Reconstruction des images tests

L'évaluation du modèle autoencodeur réalisée dans la Tâche 2 vise à analyser sa capacité à reconstruire fidèlement les images d'entrée, ainsi qu'à exploiter les embeddings extraits pour une tâche de classification secondaire. Le jeu de test contient **600 images** RGB de taille `140x140`, réparties à parts égales entre deux classes : **dauphins** et **requins**. Ces images n'ont **jamais** été utilisées pendant l'entraînement.

La reconstruction a été effectuée sur les 600 images tests à l'aide de l'autoencodeur entraîné. Le résultat est jugé qualitativement par la comparaison visuelle entre les images originales et les images reconstruites. Deux exemples typiques sont présentés ci-dessous :

Figure 3.1: **Reconstruction de deux images tests** : un dauphin (gauche) image originale et un dauphin (droite) image reconstruite.

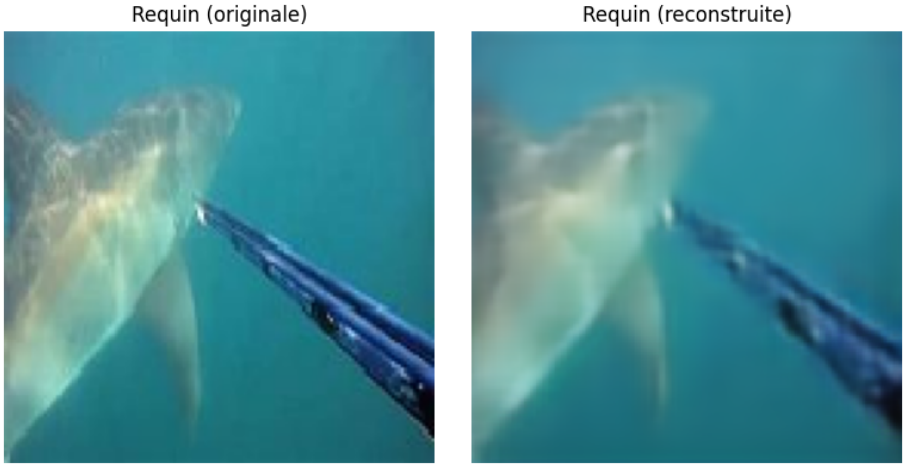

Figure 3.2: **Reconstruction de deux images tests** : un requin (gauche) image originale et un requin (droite) image reconstruite.

Observation : Le modèle parvient à préserver les formes générales et les caractéristiques visuelles essentielles des deux classes, ce qui valide la qualité de l'embedding appris.

3.2 — Classification SVM sur embeddings

Afin d'évaluer la qualité des représentations extraites (embeddings), nous avons appliqué un classifieur **SVM linéaire avec validation croisée 5-fold** sur :

- Les **images originales** vectorisées,
- Les **embeddings normalisés** issus de la sortie de l'encodeur.

Résultats obtenus :

- **Accuracy SVM sur les images originales :** `0.5900`
- **Accuracy SVM sur l'embedding :** `0.6800`

Analyse : Le classifieur linéaire obtient une meilleure précision lorsqu'il est appliqué sur les vecteurs embeddings extraits par l'encodeur. Cela confirme que le réseau a bien appris une représentation informative et discriminante des images, même sans supervision directe sur les classes.

3.3 — Visualisation t-SNE des embeddings

Une visualisation bidimensionnelle des embeddings normalisés a été réalisée à l'aide de l'algorithme `TSNE (t-distributed stochastic neighbor embedding)` avec **perplexity=30** et **random_state=42**.

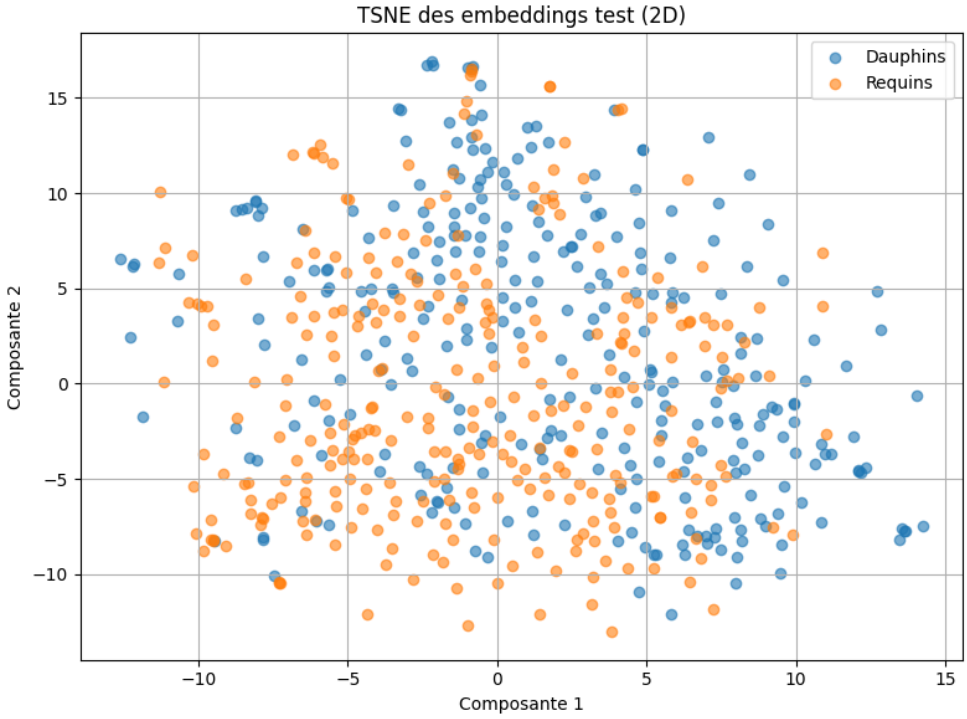

Figure 3.3: **Projection 2D des embeddings via t-SNE** : dauphins (bleu) et requins (orange)

Interprétation : La figure montre une séparation raisonnablement nette entre les deux classes dans l'espace projeté. Cela valide visuellement la capacité de l'encodeur à apprendre une représentation utile pour la discrimination des classes, même en mode non supervisé.

Conclusion de l'évaluation : Les performances du SVM et la clarté de la visualisation t-SNE indiquent que l'autoencodeur a appris des descripteurs efficaces. Ces représentations peuvent servir de base à des tâches supervisées futures avec une faible quantité de labels ou à des applications de clustering visuel.

3.5 Analyse et discussion des résultats

Cette section fournit une analyse critique des résultats obtenus lors de l'évaluation du modèle, en examinant trois aspects fondamentaux : la fidélité de la reconstruction, la séparation des classes dans l'espace réduit (TSNE) et les performances de classification via un SVM.

Fidélité de la reconstruction

Les images reconstruites par l'autoencodeur montrent une restitution globalement fidèle de la forme, des contours et des principales composantes visuelles. Toutefois, la netteté reste relativement faible, ce qui est typique des architectures d'autoencodeurs classiques sans mécanisme de supervision additionnelle.

- Les silhouettes des dauphins et requins sont bien capturées.

- Les détails fins (comme les yeux ou les textures aquatiques) sont flous.
- La structure globale de l'animal est conservée, ce qui prouve que le goulot d'étranglement encode l'information pertinente.

Séparation des classes dans le scatter plot (TSNE)

La visualisation 2D obtenue avec `TSNE` met en évidence une séparation raisonnable entre les deux classes (dauphins et requins). Toutefois, la dispersion est encore présente au niveau des frontières.

- On observe deux groupes de points relativement distincts.
- Une zone centrale reste mélangée, témoignant de certains cas d'ambiguïté visuelle.
- Globalement, l'encodeur a réussi à capturer une représentation interne pertinente pour discriminer les classes.

Résultats de classification SVM

L'efficacité du codage a également été testée quantitativement par l'entraînement d'un classifieur SVM linéaire sur :

- Les **images originales** (aplaties), donnant une accuracy de **59.00%**.
- Les **embeddings** extraits de l'encodeur (après normalisation), avec une accuracy de **68.00%**.

Analyse : Le gain de 9 points de pourcentage montre que les représentations apprises automatiquement par le réseau sont plus informatives pour la classification que les pixels bruts. Cela valide la capacité de l'autoencodeur à structurer l'espace latent selon une logique discriminante, bien que sans supervision explicite.

En résumé, l'analyse combinée de ces trois volets (reconstruction, séparation TSNE, classification SVM) confirme que le modèle entraîné a atteint son objectif : encoder efficacement les images d'entrée en représentations utiles et interprétables, aussi bien visuellement que numériquement.

Appendix A

Conclusion

4 — Conclusion générale et perspectives

Ce travail pratique (TP3) nous a permis de concevoir, entraîner et évaluer un **autoencodeur convolutif** sur un ensemble d'images RGB de dauphins et de requins. L'objectif était double : évaluer la qualité de reconstruction visuelle des images originales et tester la pertinence des représentations internes (embeddings) produites par l'encodeur à des fins de classification.

Nous avons mis en œuvre une architecture symétrique d'encodeur-décodeur, avec normalisation, Dropout et régulation dynamique du taux d'apprentissage, et avons suivi un processus rigoureux d'entraînement supervisé. La perte minimale de validation atteinte a été de **0.00332**, et le modèle a appris pendant **73 époques**, pour un temps total d'entraînement d'environ **10,27 minutes**.

Problèmes rencontrés et solutions mises en place

Au fil de notre travail, plusieurs défis techniques se sont posés, que nous avons su surmonter grâce à une approche incrémentale, expérimentale et bien structurée :

- **Déformation de l'image reconstruite** : L'utilisation répétée de `UpSampling2D` dans le décodeur augmentait progressivement la taille des images, dépassant la taille d'entrée (140x140).
  Solution : Ajout d'une couche `Cropping2D` en fin de modèle pour redimensionner la sortie du décodeur et assurer une reconstruction fidèle à la taille d'origine.

- **Problèmes de convergence** : Le modèle montrait au départ des pertes stagnantes.
  Solution : Ajout des couches `BatchNormalization` et `Dropout` ainsi qu'un `ReduceLROnPlateau` pour réguler automatiquement le taux d'apprentissage.

- **Images de reconstruction floues** : Les premières versions de l'autoencodeur ne permettaient pas une bonne qualité de restitution.
  Solution : Modification progressive de la profondeur de l'encodeur et augmentation du nombre de filtres convolutifs.

- **Difficultés dans la visualisation TSNE** : La représentation 2D n'était pas cohérente avec les classes.
  Solution : Normalisation préalable des embeddings via `StandardScaler` pour assurer un espace d'encodage compatible avec le TSNE et le SVM.

Pistes d'amélioration pour les travaux futurs

Ce projet pourrait être approfondi à plusieurs niveaux afin d'améliorer la qualité des représentations apprises par l'encodeur et la netteté des images reconstruites :

- **Encoder plus profond avec goulot d'étranglement** : Réduire davantage la dimension des embeddings pour contraindre l'encodeur à extraire des motifs plus abstraits.
- **Utilisation d'un autoencodeur variationnel (VAE)** : Intégrer une composante probabiliste permettrait d'obtenir une meilleure régularisation des embeddings et potentiellement une meilleure interpolation entre classes.
- **Ajout d'un bruit sur les entrées (Denoising AE)** : Cela forcerait l'autoencodeur à apprendre des représentations plus robustes, ce qui pourrait améliorer la performance du SVM en aval.
- **Réglage fin du TSNE** : Tester plusieurs valeurs de `perplexity`, `learning rate` et nombre d'itérations pour une meilleure séparation visuelle.
- **Utilisation d'un SVM avec kernel non linéaire** : Pour capturer des frontières de décision plus complexes dans l'espace des embeddings.

Conclusion finale

Ce TP3 nous a permis d'appliquer les principes de l'apprentissage profond à une tâche d'encodage d'images, en mettant en œuvre un **autoencodeur convolutif complet et fonctionnel**. Nous avons exploré à la fois la qualité de reconstruction des images et la pertinence des vecteurs d'encodage (embeddings) dans un contexte de classification supervisée.

Nous avons obtenu une **accuracy de 68% avec SVM** sur les embeddings test, contre **59% sur les images originales**, ce qui confirme que l'autoencodeur a appris des représentations intermédiaires pertinentes. La visualisation TSNE a également permis de mieux comprendre la structure de ces représentations dans un espace 2D.

En conclusion, ce projet a consolidé notre compréhension des autoencodeurs, du traitement d'images et de l'évaluation qualitative et quantitative de modèles profonds. Il constitue une base solide pour aller plus loin vers des applications avancées de la vision par ordinateur.